

Lecture Review: Foundations of Data Science

Yingming PU

LINs Lab, Westlake University

September 7, 2026



Outline

① Machine Learning

② Linear Algebra

③ Calculus

④ Probability Theory

⑤ Summary

The supervised pipeline in one frame

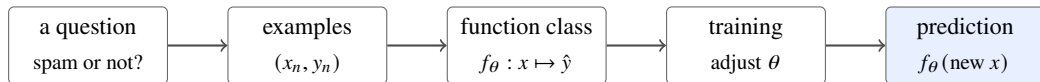


The supervised pipeline in one frame



- **Data:** $\mathcal{D} = \{(x_n, y_n)\}_{n=1}^N, x_n \in \mathbb{R}^D$.
- **Model:** $\hat{y} = f_\theta(x)$; inference uses the trained f_θ only.
- **Training:** $\theta \leftarrow \text{opt}(\theta, \nabla \ell)$ on $\ell(\hat{y}_n, y_n)$.
- **Evaluation:** held-out data, distribution shifts, deployment costs.

From a real-world question to a learning problem



Machine learning replaces “**write the rules by hand**” with “**estimate a function from examples**”.

The question, the data, and the evaluation are still chosen by people.

A model is just a function; training is choosing its parameters

```
def f(x, theta):  
    # numbers in, numbers out  
    return x @ theta    # e.g. a linear map  
  
def loss(theta, data): # how wrong f is  
    return sum((f(x, theta) - y)**2  
               for x, y in data)  
  
# training = make loss(theta) small
```

- **Prediction** = evaluating a function.
- **Training** = choosing θ to make $\text{loss}(\theta)$ small on examples.
- “Deep” only means f is a *composition* of many simple functions.

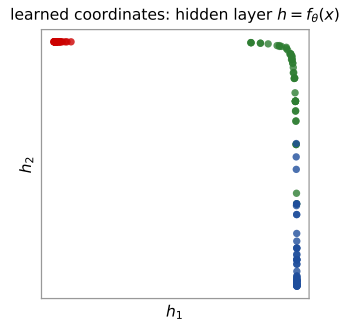
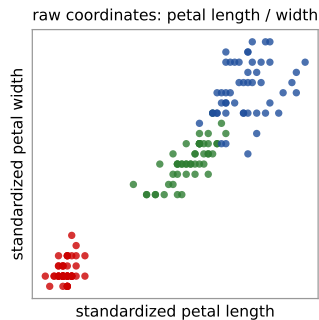
A model is just a function; training is choosing its parameters

```
def f(x, theta):  
    # numbers in, numbers out  
    return x @ theta    # e.g. a linear map  
  
def loss(theta, data): # how wrong f is  
    return sum((f(x, theta) - y)**2  
               for x, y in data)  
  
# training = make loss(theta) small
```

- **Prediction** = evaluating a function.
- **Training** = choosing θ to make $\text{loss}(\theta)$ small on examples.
- “Deep” only means f is a *composition* of many simple functions.

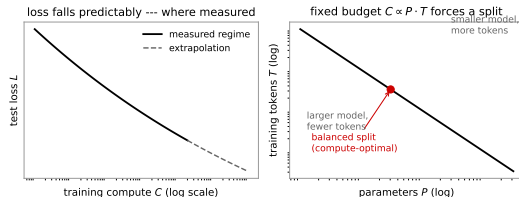
There is no mystery inside the box: a model is a parameterized function, and learning is parameter estimation from data.

A learned representation is a loss-selected change of coordinates



“Learning a representation” = Re-parameterizing the input so that the task becomes a simple geometric problem.

Why bigger models keep helping — and what a budget forces you to choose



- ① More compute $\Rightarrow$ test loss falls along a smooth, predictable curve.
- ② Budget constraint $C \propto P \cdot T$: bigger model P or more tokens T — a fixed budget forces a split.
- ③ Compute-optimal runs split the budget near-equally (Hoffmann et al., 2022).

Scaling laws are empirical fits: they predict where they were measured; beyond that regime, the curve is a hypothesis.

Outline

① Machine Learning

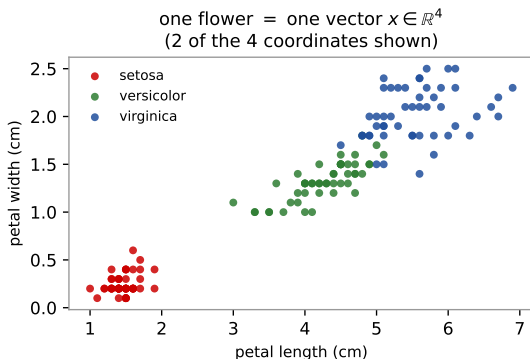
② Linear Algebra

③ Calculus

④ Probability Theory

⑤ Summary

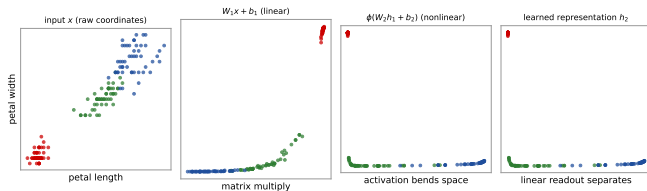
Why linear algebra: data already arrives as vectors and matrices



- One Iris flower = four measurements = one vector $x \in \mathbb{R}^4$.
- 150 flowers = one matrix $X \in \mathbb{R}^{150 \times 4}$.
- Classifying the species = finding a geometric rule (e.g. a separating direction) in $\mathbb{R}^4$.

Measurements are vectors and datasets are matrices; machine-learning questions become questions about points, directions, and subspaces. This is what linear algebra is *for* in this course.

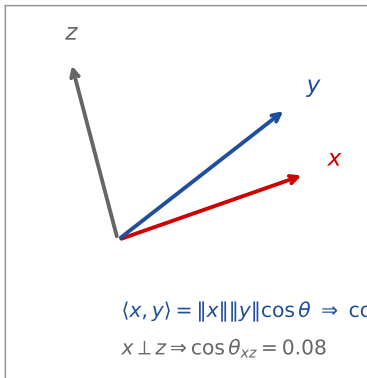
MLP layers learn coordinates that separate Iris



- Each matrix multiplies and mixes coordinates.
- Linear layers alone collapse to one matrix: W_2W_1x is still linear.
- The activation ϕ bends the space; this is where new separability can appear.

MLP = alternating linear coordinate changes and nonlinear bending; the final coordinates are selected by classification loss.

The inner product measures similarity as alignment

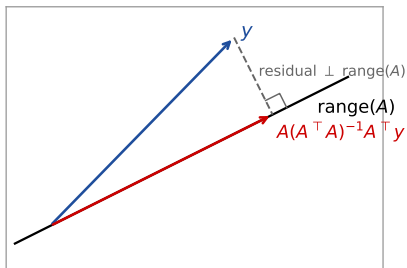


- $\langle x, y \rangle = \|x\|_2 \|y\|_2 \cos \theta$: large when directions align, zero when orthogonal, negative when opposed.
- Normalized ($\cos \theta$ alone) = cosine similarity, the standard score for comparing embedding vectors.

Similarity between two states is the alignment of their directions; dividing by the norms removes scale and keeps only the direction.

Projection: the best approximation a subspace allows

$$\text{Proj}_{\text{range}(A)}(y) = \arg \min_{v \in \text{range}(A)} \|v - y\|_2 = A(A^\top A)^{-1}A^\top y$$

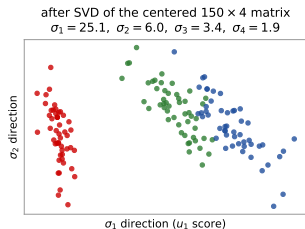
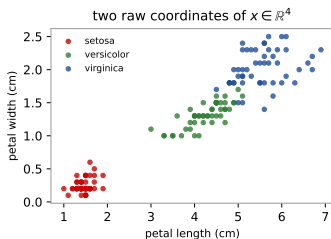


- Closed form from the normal equations $(A^\top A)w = A^\top y$; full column rank required, otherwise $AA^\dagger y$.
- The residual is orthogonal to $\text{range}(A)$ (right-angle marker in the figure).

Least-squares regression *is* this projection; structure of y outside $\text{range}(X)$ is unrecoverable.

SVD finds the directions that carry the data (Iris, measured)

$$A = \sum_{i=1}^r \sigma_i u_i v_i^\top$$



- Term i : measure along v_i , scale by σ_i , emit along u_i ; terms act independently.
- On centered Iris: $\sigma_1=25.1$ dominates $\sigma_{2..4}=(6.0, 3.4, 1.9)$; two SVD directions separate the classes better than any two raw coordinates.

$\sigma_1 \geq \sigma_2 \geq \dots$ ranks directions by how much of the data they carry; truncation keeps the strongest k (PCA). Calling the discarded tail “noise” is a modeling assumption, not a theorem.

Outline

① Machine Learning

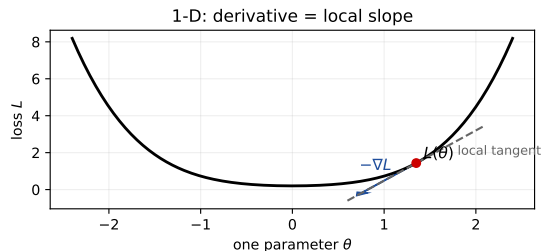
② Linear Algebra

③ Calculus

④ Probability Theory

⑤ Summary

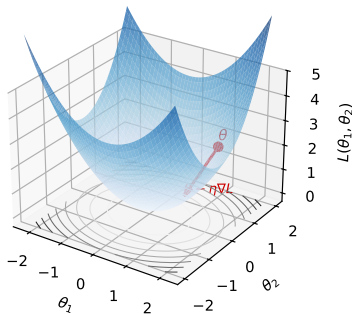
Steepest descent, derived rather than asserted



∇L is the local slope; step in $-\nabla L$ to reduce loss.

Gradient descent on a 3-D loss surface

3-D loss surface and one descent step



$$\nabla L(\theta) = \begin{bmatrix} \partial L / \partial \theta_1 \\ \partial L / \partial \theta_2 \end{bmatrix}, \quad \theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)$$

The gradient is perpendicular to a level curve; repeated negative-gradient steps descend toward a minimum.

Outline

- 1 Machine Learning
- 2 Linear Algebra
- 3 Calculus
- 4 Probability Theory
- 5 Summary

A random variable is a deterministic observation function

- $(\Omega, \mathcal{F}, P)$: the sample space Ω is typically too complex to model directly.

A random variable is a deterministic observation function

- $(\Omega, \mathcal{F}, P)$: the sample space Ω is typically too complex to model directly.
- $X : \Omega \rightarrow \mathbb{R}$ assigns a number to each outcome; we model the induced law, not Ω :

$F_X(x) \triangleq P(X \leq x)$ (CDF); p_X (PMF, countable support); f_X (PDF, probabilities = areas).

A random variable is a deterministic observation function

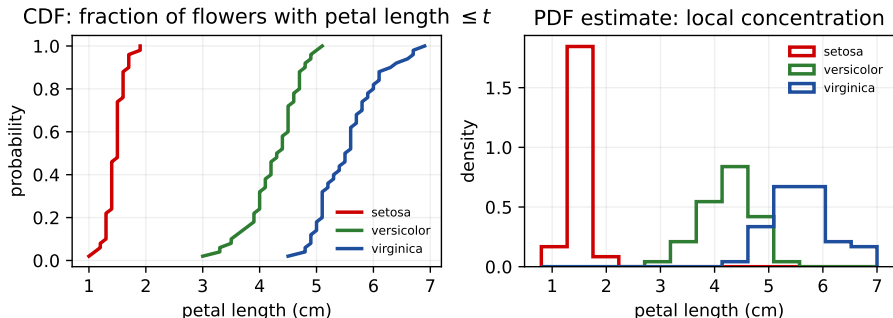
- $(\Omega, \mathcal{F}, P)$: the sample space Ω is typically too complex to model directly.
- $X : \Omega \rightarrow \mathbb{R}$ assigns a number to each outcome; we model the induced law, not Ω :

$F_X(x) \triangleq P(X \leq x)$ (CDF); p_X (PMF, countable support); f_X (PDF, probabilities = areas).

- Density height $\neq$ point probability: f_X can exceed 1 (Uniform($0, \frac{1}{2}$) has height 2, area 1).

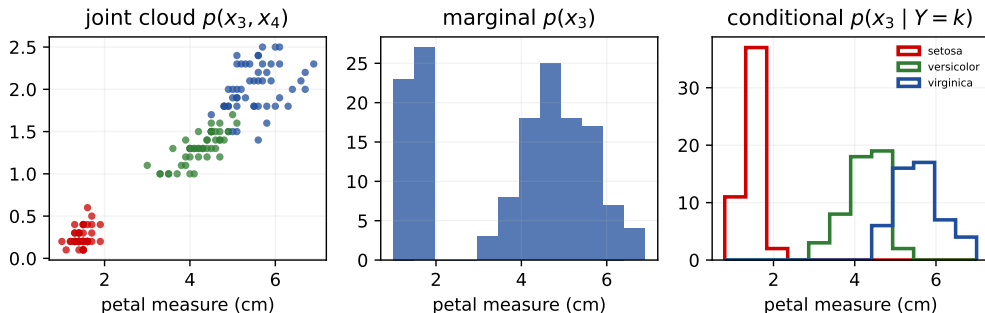
All probabilistic statements about X are statements about P on Ω , pulled back through a deterministic map; the distribution is the only handle we model.

CDF and PDF: two views of the same Iris measurement



CDF answers “how much has accumulated by t ?”; PDF answers “where is mass concentrated?”
Probability is an area (or CDF difference), not a bar height.

Joint, marginal, and conditional distributions



Joint keeps relationships between measurements; marginal forgets the other coordinate;
conditioning isolates a context such as species.

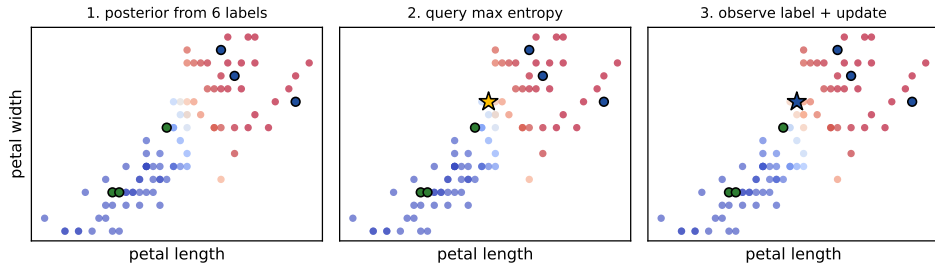
Bayes's rule: an algebraic identity that reverses inference

$$p_{XY}(x, y) = p_{Y|X}(y|x) p_X(x) = p_{X|Y}(x|y) p_Y(y) \implies p_{Y|X}(y|x) = \frac{p_{X|Y}(x|y) p_Y(y)}{\sum_{y' \in \mathcal{Y}} p_{XY}(x, y')}, \quad p_X(x) > 0.$$

- Generative direction: latent $Y \rightarrow$ observation X (likelihood $p_{X|Y}$, prior p_Y).
- Inference direction: observation $\rightarrow$ latent (posterior $p_{Y|X}$).
- Precondition: every conditioning denominator strictly positive.

The posterior is a deterministic consequence of prior, likelihood, and evidence — the rule itself involves no approximation; all assumptions enter through the choice of prior and likelihood.

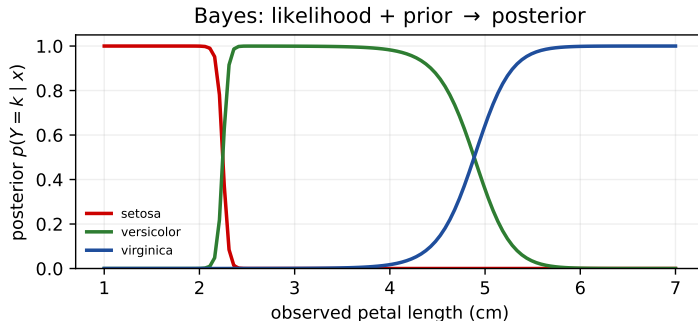
Bayes enables active learning



$$x^* = \arg \max_{x \in \mathcal{U}} H[p(y | x, \mathcal{D})], \quad H(p) = - \sum_y p(y) \log p(y)$$

Query the most uncertain unlabeled flower, observe its label, and update the posterior.

Bayes on Iris: evidence updates a species belief I



- Prior: before measuring, each species has probability $1/3$.
- Likelihood: how compatible is this petal length with each species?
- Posterior: normalize the three likelihoods; this is the classifier's belief.

Bayes on Iris: evidence updates a species belief II

Bayes is the bookkeeping rule that turns an observation into a calibrated belief about an unobserved cause

Outline

- 1 Machine Learning
- 2 Linear Algebra
- 3 Calculus
- 4 Probability Theory
- 5 Summary

Three checks to apply to any problem in this course

- 1 **Dimensions and subspaces** (linear algebra): name the space of every object, verify that inner dimensions match, and state which subspace an operation retains or discards (range, nullspace, rank, truncated SVD).

Three checks to apply to any problem in this course

- 1 **Dimensions and subspaces** (linear algebra): name the space of every object, verify that inner dimensions match, and state which subspace an operation retains or discards (range, nullspace, rank, truncated SVD).
- 2 **Distributions and conditioning** (probability): state which variables are observed and which are latent, check that every conditioning denominator is positive, and marginalize latent variables.

Three checks to apply to any problem in this course

- 1 **Dimensions and subspaces** (linear algebra): name the space of every object, verify that inner dimensions match, and state which subspace an operation retains or discards (range, nullspace, rank, truncated SVD).
- 2 **Distributions and conditioning** (probability): state which variables are observed and which are latent, check that every conditioning denominator is positive, and marginalize latent variables.
- 3 **Gradients of compositions** (calculus): write the model as a composition and compute parameter gradients by Jacobian-transpose–vector products with matching shapes.

Lecture 1 defines one recurring structure: data and model define a loss over parameters; linear algebra specifies the spaces in which data, parameters, and gradients live; calculus supplies the update direction and the chain rule that computes it; probability supplies the objectives — likelihoods, posteriors, expectations — that the loss instantiates.